



Infrastructure & Security Portfolio

Cloud Infrastructure · IaC · Security

Monitoring

Cloud Infrastructure · IaC · Security Monitoring

PROJECT 01

Terraform 기반 AWS 인프라 구축

03

Infrastructure · IaC · Cloud · Security

04

프로젝트 개요

05

문제 정의

06

구축 아키텍처

07

구축 과정

08

프로젝트 성과

09

배운 점

PROJECT 02

Suricata·ELK 기반 IDS 구축

10

Web Attack Detection · Log Analysis · Monitoring

11

프로젝트 개요

12

문제 정의

13

구축 아키텍처

14

구축 과정

15

프로젝트 성과

16

배운 점

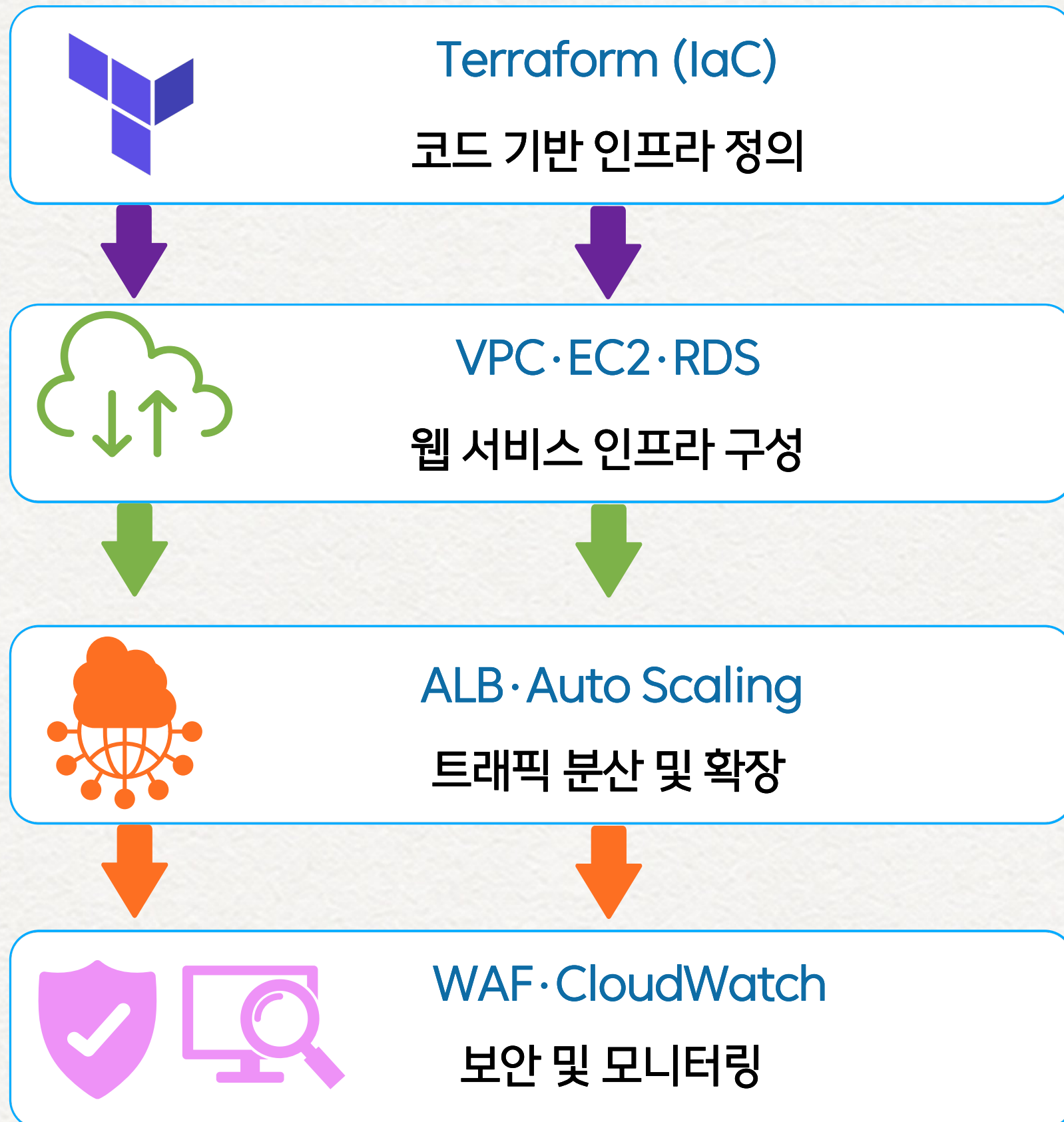


Terraform 기반 AWS 인프라 구축 Portfolio

Infrastructure · IaC · Cloud · Security

Terraform 기반 AWS 웹 서비스 인프라 구축

(Terraform | AWS | IaC)



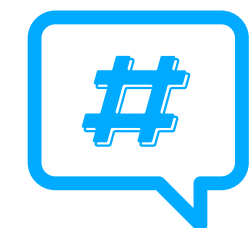
프로젝트 목적

코드 기반으로 재현 가능한 AWS 웹 서비스 인프라를 구축하고, 트래픽 대응 · 보안 · 모니터링까지 고려한 안정적인 운영 환경을 구현하는 것



프로젝트 기간

2025.12.17 ~ 2026.01.02



핵심 키워드

#Terraform #AWS #IaC
#Web Service #Auto Scaling
#WAF #CloudWatch

Cloud (AWS)



문제 인식

수작업 기반 AWS 인프라 구축의 한계

기존 AWS 환경에서는 리소스를 개별적으로 생성·설정하여 반복 작업과 설정 오류 가능성이 있었고, 동일 환경의 재현 및 변경 관리에도 어려움이 있었습니다.

해결 방향

Terraform 기반 IaC 적용

AWS 리소스를 Terraform 코드로 정의하여 동일한 인프라를 재현하고 변경 사항을 일관되게 관리하고자 했습니다.

수작업 구축

반복 작업 · 설정 오류 가능성 · 낮은 재현성 · 변경 관리 부담



Terraform IaC 적용

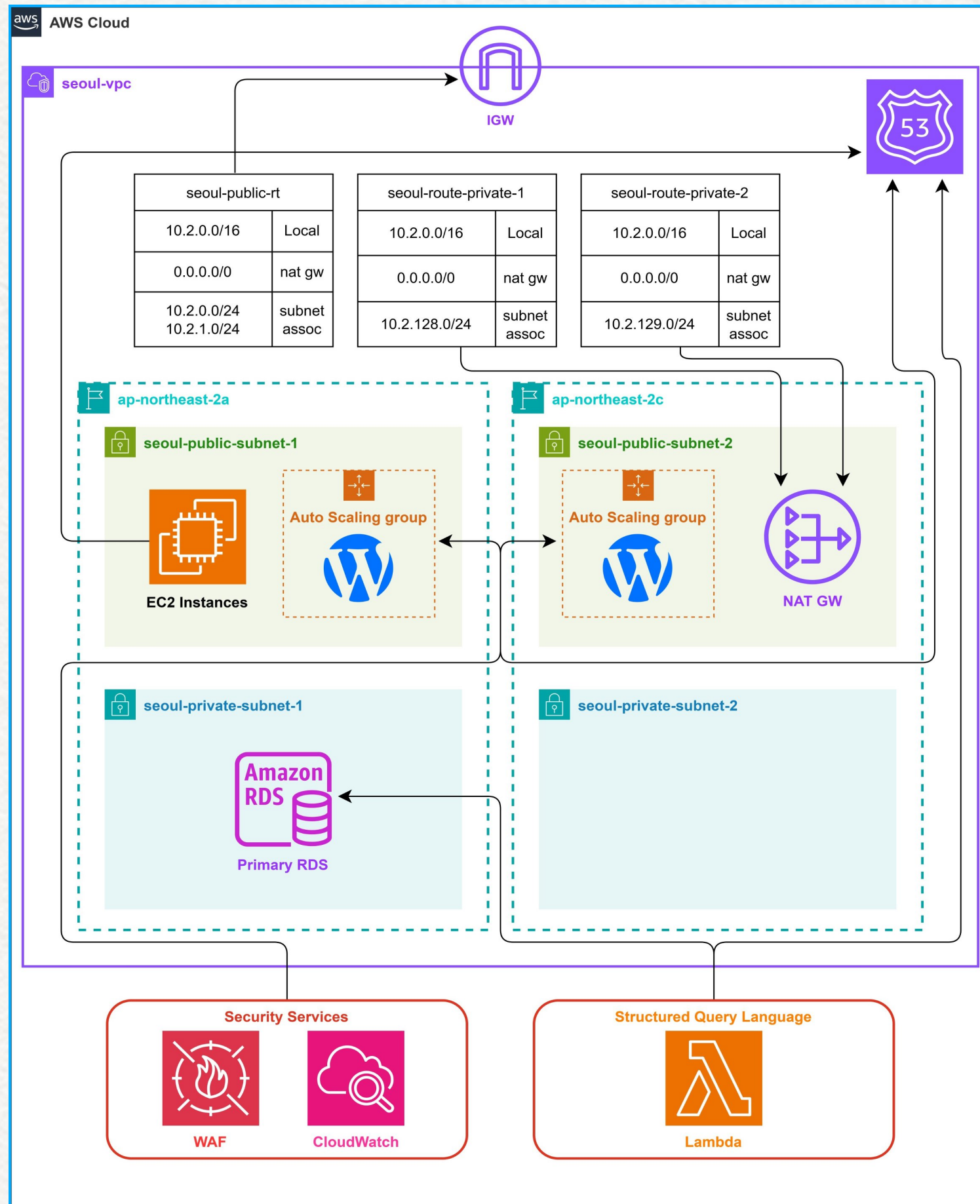
코드 기반 인프라 구축

재현성 · 일관성 · 자동화 · 변경 관리

구축 아키텍처

(Terraform 기반 AWS 웹 서비스 인프라 구성)

AWS Architecture



아키텍처 구성

01. 네트워크 영역 분리

Public · Private Subnet을 분리하여 웹 서비스와 데이터베이스 영역 구성

02. 웹 서비스 및 트래픽 처리

EC2 · ALB · Auto Scaling을 연계하여 트래픽 분산 및 자동 확장 구성

03. 데이터베이스 구성

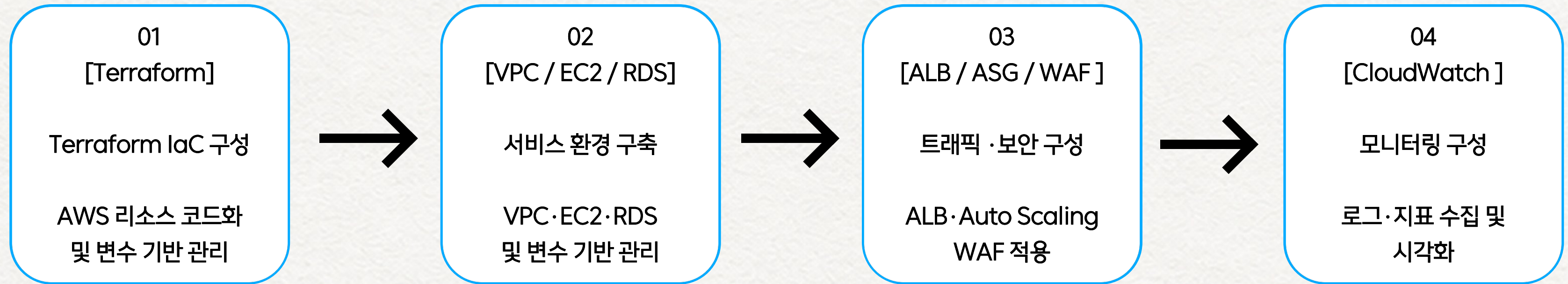
RDS를 Private Subnet에 배치하여 외부 직접 접근 제한

04. 보안 및 모니터링

AWS WAF를 통한 웹 요청 보호 및 CloudWatch 기반 모니터링 구성

Terraform으로 주요 AWS 리소스를 코드화하여 동일한 인프라 구성을 재현할 수 있도록 설계했습니다.

Cloud 기반 IaC 구축 과정



IaC 구현
[Terraform 코드]

```
terraform {  
  required_version = "~> 1.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 6.0"  
    }  
  }  
}  
  
locals {  
  conf = yamldecode(file("./variables.yaml"))  
}  
  
provider "aws" {  
  region = local.conf.region  
}
```

모니터링 결과
[CloudWatch Dashboard]





CloudWatch Dashboard

```
=====
WAF Attack Test Started
Target: http://www.siu.it-edu.org
=====

[1] SQL Injection Attack Test
Attack: http://www.siu.it-edu.org?id=1' OR '1'='1
[OK] Blocked by WAF (403 Forbidden)
Attack: http://www.siu.it-edu.org?id=1' UNION SELECT NULL--
[OK] Blocked by WAF (403 Forbidden)
Attack: http://www.siu.it-edu.org?id=1; DROP TABLE users--
[OK] Blocked by WAF (403 Forbidden)
Attack: http://www.siu.it-edu.org?id=1' OR 1=1--
[OK] Blocked by WAF (403 Forbidden)
Attack: http://www.siu.it-edu.org/wp-login.php?user=admin'--
[OK] Blocked by WAF (403 Forbidden)

[2] WordPress Specific Attack Test
Attack: http://www.siu.it-edu.org/wp-admin/admin-ajax.php?action=revslider_show_image&img=../wp-config.php
[OK] Blocked by WAF (403 Forbidden)

[3] Rate-based Brute Force Attack Test
Path: /wp-login.php (Blocked if 15+ requests in 5 min)
Sending 16 requests quickly...
Request #1... Allowed (HTTP 200)
Request #2... Allowed (HTTP 200)
Request #3... Allowed (HTTP 200)
Request #4... Allowed (HTTP 200)
Request #5... Allowed (HTTP 200)
Request #6... Allowed (HTTP 200)
Request #7... Allowed (HTTP 200)
Request #8... Allowed (HTTP 200)
Request #9... Allowed (HTTP 200)
Request #10... Allowed (HTTP 200)
Request #11... Allowed (HTTP 200)
Request #12... Allowed (HTTP 200)
Request #13... Allowed (HTTP 200)
Request #14... Allowed (HTTP 200)
Request #15... Allowed (HTTP 200)
Request #16... Allowed (HTTP 200)

Result: Allowed 16 times, Blocked 0 times

=====
WAF Attack Test Completed
=====
```

WAF Attack Test



프로젝트 성과

01. 코드 기반 인프라 구축

Terraform을 활용해 AWS 리소스를 코드로 정의하고, 동일한 인프라 구성을 반복적으로 재현할 수 있는 환경을 구현했습니다.

02. 트래픽 대응 및 웹 보안 구성

ALB와 Auto Scaling을 연계하여 트래픽 분산 및 자동 확장 구조를 구성하고, AWS WAF를 적용하여 웹 공격 차단 동작을 검증했습니다.

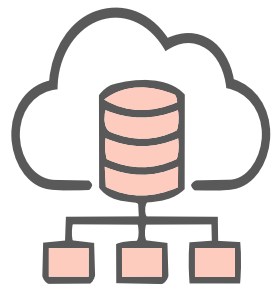
03. 모니터링 환경 구현

CloudWatch Dashboard를 구성하여 웹 서비스의 로그와 주요 지표를 확인할 수 있는 모니터링 환경을 구현했습니다.



1 IaC의 목적과 운영 관점

Terraform을 활용해 AWS 리소스를 코드로 정의하며, IaC는 단순히 구축 작업을 자동화하는 것이 아니라 동일한 환경을 재현하고 변경 사항을 일관되게 관리하기 위한 방식이라는 점을 경험했습니다.



2 서비스 운영을 고려한 인프라 설계

VPC, EC2, RDS, ALB, Auto Scaling 등을 연계하며 개별 리소스의 구성에 그치지 않고 서비스의 트래픽 흐름과 네트워크·데이터베이스 구조를 함께 고려하여 인프라를 설계하는 것이 중요하다는 점을 배웠습니다.



3 보안과 모니터링까지 고려한 구축

AWS WAF와 CloudWatch를 구성하고 실제 동작을 확인하면서, 인프라는 서비스 구축뿐만 아니라 보안 정책 적용과 운영 상태를 지속적으로 확인할 수 있는 환경까지 함께 고려해야 한다는 점을 배웠습니다.



Suricata · ELK 기반 IDS 구축 Portfolio

Web Attack Detection · Log Analysis · Monitoring

Suricata·ELK 기반 IDS 모니터링 환경 구축

(Rocky Linux | Suricata | ELK)



Suricata

웹 공격 트래픽 탐지 및 분석



Filebeat

탐지 이벤트 수집 및 전달



Elasticsearch

이벤트 저장 및 검색



Kibana

로그 분석 및 시각화



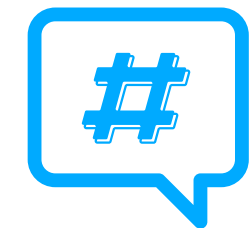
프로젝트 목적

웹 공격 트래픽을 탐지하고 보안 이벤트를 수집·분석할 수 있는 IDS 모니터링 환경을 구축하여 보안 운영의 효율성과 안정성을 확보하는 것



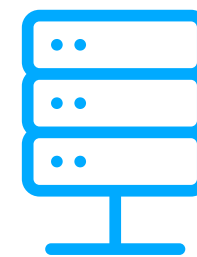
프로젝트 기간

2025.12.17 ~ 2026.01.02



핵심 키워드

#Suricata #ELK #IDS
#Filebeat #Elasticsearch #Kibana
#Log Analysis #Monitoring



핵심 환경

Rocky Linux Suricata
ELK(Elasticsearch, Kibana) Filebeat



문제 인식

분산된 로그 환경의 한계

웹·DNS·DB 서버의 로그가 분산되어 있어 공격 이벤트를 통합 확인하기 어렵고, 수동 로그 확인으로는 실시간 탐지·분석에 한계가 있었습니다.

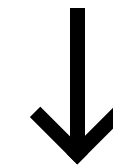
해결 방향

Suricata·ELK 기반 IDS 모니터링 환경 구축

Suricata에서 공격을 탐지하고 Filebeat를 통해 이벤트를 Elasticsearch로 전달하여, Kibana에서 검색·분석·시각화할 수 있도록 구성했습니다.

기존 환경

로그 분산 · 수동 확인 · 실시간 탐지 어려움 · 통합 분석 어려움



Suricata·ELK 연계

개선 환경

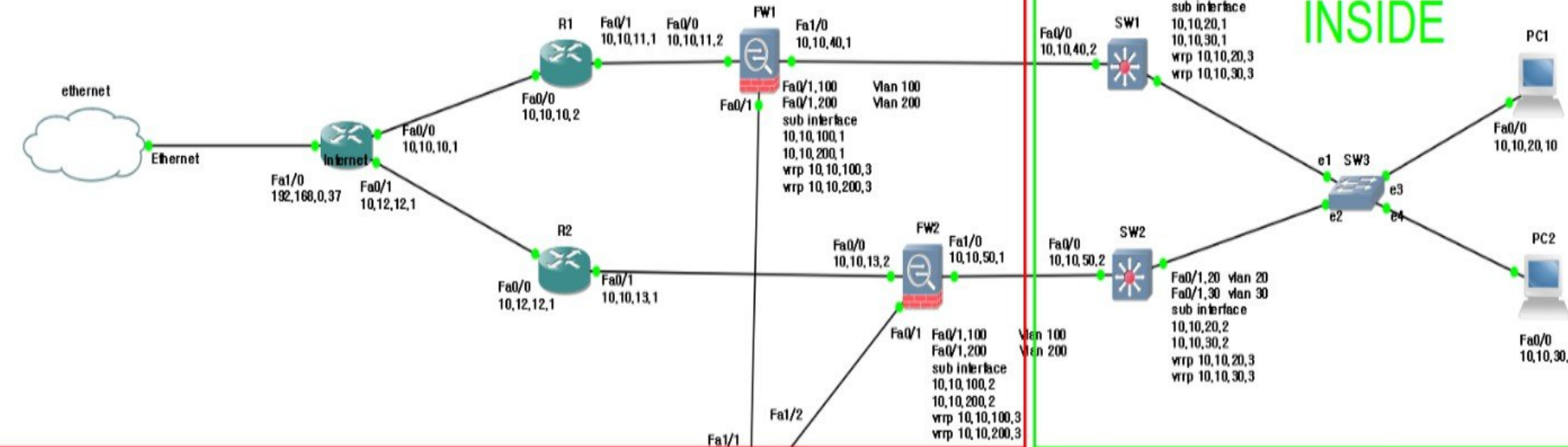
공격 탐지 · 이벤트 중앙 수집 · 로그 검색·분석 · 시각화

구축 아키텍처

(Suricata·ELK 기반 IDS 모니터링 환경 구성)

On-Premise (IDS)

OUTSIDE



INSIDE

SURIKATA
10.10.100.33
10.10.200.33

아키텍처 구성

01. 네트워크 영역 분리

Outside · Inside · DMZ 영역을 구분하여 외부 트래픽과 내부 서비스 영역 분리

02. 공격 탐지 환경 구성

Suricata IDS를 구성하고 탐지 Rule을 적용하여 웹 공격 이벤트 탐지

03. 이벤트 수집 및 저장

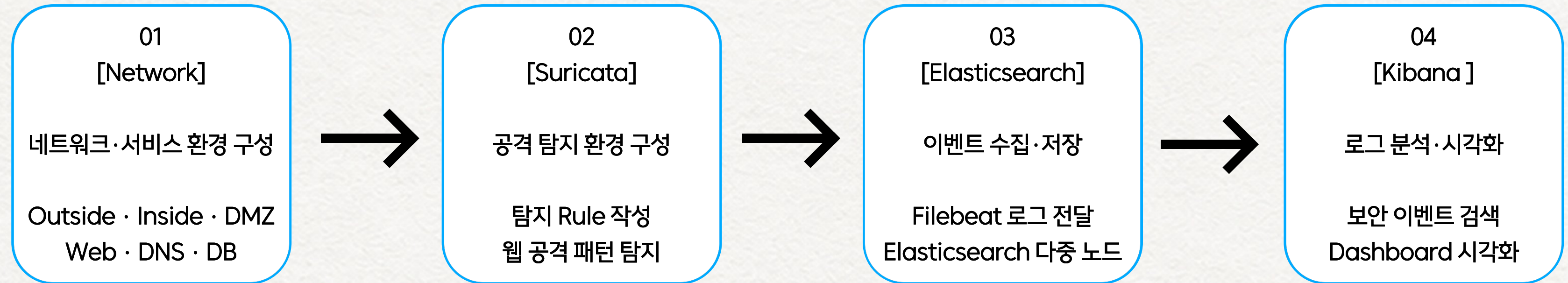
Filebeat로 Suricata eve.json 이벤트를 전달하고 Elasticsearch 다중 노드에 저장

04. 로그 분석 및 시각화

Kibana를 연동하여 보안 이벤트 검색 · 분석 및 Dashboard 시각화

Suricata → Filebeat → Elasticsearch → Kibana 연계를 통해 공격 탐지부터 수집·저장·분석·시각화까지 하나의 흐름으로 구성

On-Premise IDS 구축 과정



```
1 Rocky 9 - suricata x +
Brute Force -----
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"[Brute Force Attempt]"; sid:2000104; gid:2000104; rev:1; content:"GET"; http_method; nocase; content:"/vulnerabilities/brute?"; http_uri; nocase; detection_filter:track by_src, count 3, seconds 5; classtype:attempted-admin;)
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"[Brute Force Success]"; sid:2000105; gid:2000105; rev:1; content:"Welcome to the password protected area"; nocase; classtype:successful-admin;)
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"[Brute Force Failure]"; sid:2000106; gid:2000106; rev:1; content:"Username and/or password incorrect"; nocase; classtype:attempted-admin;)
```

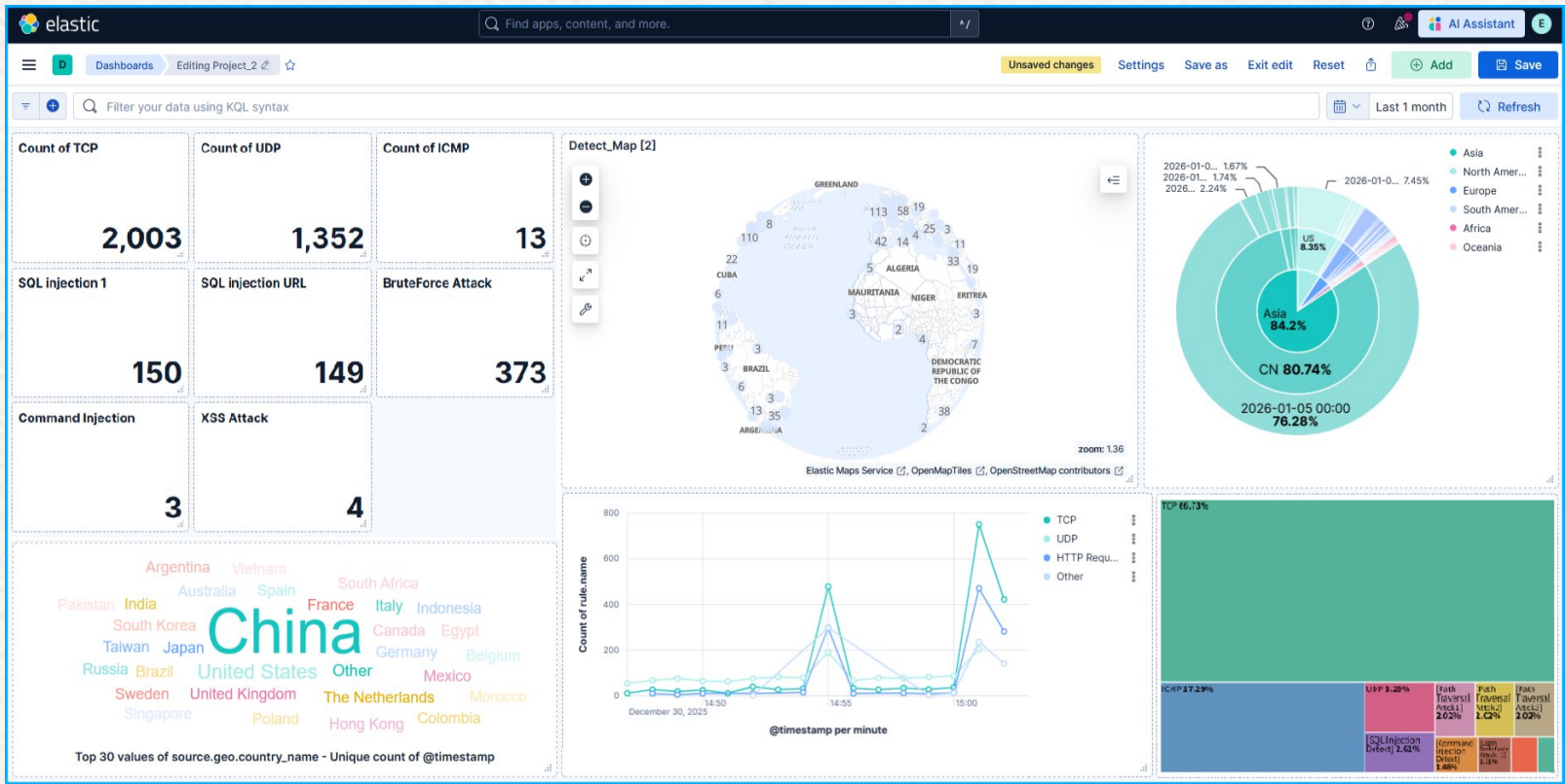
Suricata 탐지 Rule 작성

```
elastic2 x +
[root@node2 ~]# curl --cacert /etc/elasticsearch/certs/http_ca.crt \
-u 'elastic:2jYzoBkrSlftXTcn+k2w' \
https://node1.siu.it.kr:9200/
{
  "name" : "node1.siu.it.kr",
  "cluster_name" : "siu.it.kr",
  "cluster_uuid" : "jEbbKosAQnQb7iv9Xy_uRg",
  "version" : {
    "number" : "9.2.2",
    "build_flavor" : "default",
    "build_type" : "rpm",
    "build_hash" : "ed771e6976fac1a085affabd45433234a4babeaf",
    "build_date" : "2025-11-27T08:06:51.614397514Z",
    "build_snapshot" : false,
    "lucene_version" : "10.3.2",
    "minimum_wire_compatibility_version" : "8.19.0",
    "minimum_index_compatibility_version" : "8.0.0"
  },
  "tagline" : "You Know, for Search"
}
```

Elasticsearch 3Node Cluster 구축


```
[root@suricata ~]# tail -n 0 -f /var/log/suricata/fast.log
01/06/2026-12:02:05.722139  [**] [2000101:2000101:1] [Login Attempt] [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.1.180:62966 -> 10.10.100.22:80
01/06/2026-12:02:10.974004  [**] [2000101:2000101:1] [Login Attempt] [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.1.180:62973 -> 10.10.100.22:80
01/06/2026-12:02:13.378864  [**] [2000101:2000101:1] [Login Attempt] [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.1.180:62973 -> 10.10.100.22:80
01/06/2026-12:02:15.149243  [**] [2000101:2000101:1] [Login Attempt] [**] [Classification: Attempted Administrator Privilege Gain] [Priority: 1] {TCP} 192.168.1.180:62973 -> 10.10.100.22:80
```

Suricata Alert



Kibana Dashboard



프로젝트 성과

01. 웹 공격 탐지 환경 구현

Suricata 탐지 Rule을 적용하여 SQL Injection, XSS, Brute Force 등의 웹 공격을 재현하고 보안 이벤트가 정상적으로 탐지되는 것을 확인했습니다.

02. 보안 이벤트 수집·분석 환경 구현

Suricata에서 발생한 이벤트를 Filebeat를 통해 Elasticsearch에 수집하고, Kibana Dashboard에서 공격 유형과 발생 현황을 분석·시각화할 수 있도록 구성했습니다.

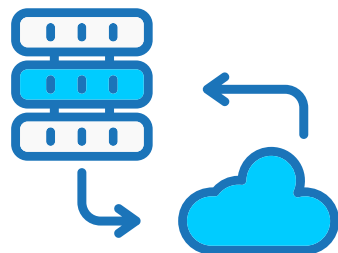
03. 로그 저장 환경의 가용성 고려

Elasticsearch를 다중 노드 클러스터로 구성하여 단일 노드 장애를 고려한 로그 저장 구조를 구현하고, 보안 이벤트를 지속적으로 저장·분석할 수 있는 환경을 구성했습니다.



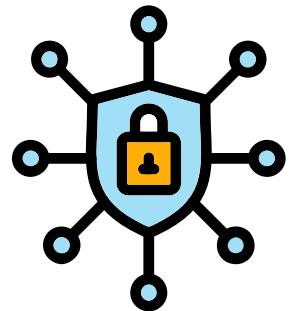
1 탐지에서 분석까지 이어지는 보안 운영

Suricata로 공격을 탐지하는 것에서 끝나는 것이 아니라, Filebeat를 통해 이벤트를 수집하고 Elasticsearch에 저장한 뒤 Kibana에서 분석·시각화하는 과정을 경험했습니다. 이를 통해 보안 환경에서는 탐지 이후 로그를 분석하고 운영에 활용하는 과정이 중요하다는 점을 배웠습니다.



2 가용성을 고려한 로그 저장 구조

Elasticsearch를 다중 노드 클러스터로 구성하며 로그 분석 환경에서도 장애를 고려한 저장 구조와 가용성 설계가 중요하다는 점을 배웠습니다. 단순히 로그를 저장하는 것을 넘어 안정적으로 분석 환경을 운영하기 위한 구조를 고려하는 경험을 할 수 있었습니다.



3 네트워크와 보안을 연결한 환경 구성

Outside·Inside·DMZ 영역을 구분하고 웹·DNS·DB 서비스와 IDS 환경을 연계하면서, 보안 시스템은 개별 솔루션만으로 구성되는 것이 아니라 네트워크와 서비스 구조를 함께 이해해야 한다는 점을 배웠습니다.